

Orion Order Platform — Technical Documentation

Demo document for Retrieval-Augmented Generation (RAG) pipeline testing.

1. Architecture Overview

The Orion Order Platform is a fictional backend system for processing customer orders. It uses FastAPI for HTTP APIs, PostgreSQL for persistent data, Redis for caching, and Apache Kafka for asynchronous event processing.

The platform is divided into API, Order Service, Payment Service, Inventory Service, Notification Service, and Analytics components.

2. Request Flow

A client sends an HTTP request to the FastAPI API layer. The API validates the request and stores the initial order in PostgreSQL.

After the order is created, the Order Service publishes an OrderCreated event to Kafka. Inventory and Payment consumers process the event independently.

Once payment and inventory checks succeed, the order state changes to Confirmed. A notification event is then published for the Notification Service.

3. API Endpoints

POST /orders creates a new order. Required fields include customer_id and items. A successful request returns HTTP 201 and an order identifier.

GET /orders/{order_id} retrieves the current order state. It returns HTTP 404 when the order does not exist.

POST /orders/{order_id}/cancel requests cancellation. Cancellation is rejected when the order has already entered a non-cancellable fulfillment stage.

4. Kafka Topics

The order.created topic contains events generated after successful order creation. The payment.result topic contains payment processing outcomes.

The inventory.result topic contains inventory reservation outcomes. The notification.requested topic contains messages that should be delivered to customers.

Kafka consumers should commit offsets only after successfully processing the corresponding message. Failed messages may be retried according to the configured retry policy.

5. Database Design

The orders table contains id, customer_id, status, total_amount, currency, created_at, and updated_at. The order_items table contains id, order_id, product_id, quantity, and unit_price.

An order can have many order items. The order_items.order_id column references orders.id.

Indexes should be created for frequently queried fields such as `orders.customer_id`, `orders.status`, and `orders.created_at`.

6. Redis Usage

Redis is used for short-lived caching of product information and selected order summaries. Cached values should have an expiration time to prevent stale data from remaining indefinitely.

The recommended cache key format for an order summary is `order:summary:{order_id}`. Cache invalidation should occur whenever important order fields change.

7. Error Handling and Observability

The API should return structured JSON errors containing an error code, human-readable message, and request identifier.

Health endpoints should distinguish between process health and dependency health. A service can be running while PostgreSQL or Kafka is unavailable.

Important metrics include request latency, HTTP error rate, Kafka consumer lag, database connection utilization, and order-processing failure rate.

8. Deployment

The platform can be packaged into Docker containers and deployed on Kubernetes. Configuration should be supplied through environment variables or Kubernetes Secrets and ConfigMaps.

Production deployments should use readiness and liveness probes. Readiness should fail when the service cannot safely receive traffic, while liveness should indicate whether the process itself is functioning.

9. Example Failure Scenarios

If Kafka becomes unavailable, the API may continue accepting requests if orders are persisted locally and event publication uses a reliable outbox or retry mechanism.

If Redis becomes unavailable, the application should fall back to the database for non-critical cached data rather than treating cache failure as total application failure.

If a consumer repeatedly fails on one message, the system should use retry limits and a dead-letter topic to prevent one poison message from permanently blocking unrelated events.